Model FSM Antarmuka OLED dan Tombol BOOT

Template firmware ESP32-C3 OLED — IoT Platform, Robotic and Embedded System Laboratory

7 November 2026

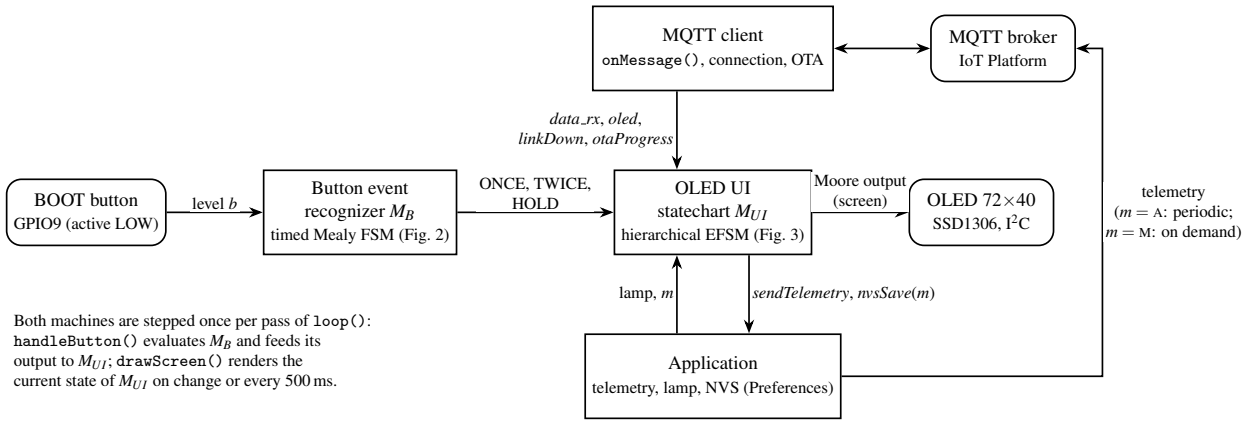
Sumber: `firmware/esp32c3_oled_mqtt/esp32c3_oled_mqtt.ino` (`readButton()`, `handleButton()`, `openMenuItem()`, `loopOled()`, `showNotice()`, `showMessage()`, `showStatus()`). Gambar dibuat dengan TikZ (`.tex`, bisa langsung di-`\input` / dikompilasi `pdflatex`); tersedia juga `.pdf` (vektor, untuk naskah LaTeX), `.svg`, dan `.png` 300 dpi.

Antarmuka ini **memang FSM**, tepatnya dua mesin yang dirangkai (*cascade*), lihat Gambar 1:

Mesin	Jenis	Masukan	Keluaran
M_B pengenal kejadian tombol	FSM Mealy berwaktu (*timed Mealy machine*)	tepi tekan/lepas ter-*debounce* dan detak waktu	ONCE, TWICE, HOLD
M_{UI} antarmuka OLED	FSM diperluas hierarkis (*hierarchical EFSM*, notasi UML *state machine*)	keluaran M_B dan kejadian MQTT	isi layar (keluaran Moore) dan aksi (Mealy)

Kedua mesin dijalankan satu langkah setiap putaran `loop()`; penjaga waktu (*guard*) dievaluasi pada setiap putaran, sehingga resolusi waktunya sama dengan periode `loop()` (orde milidetik).

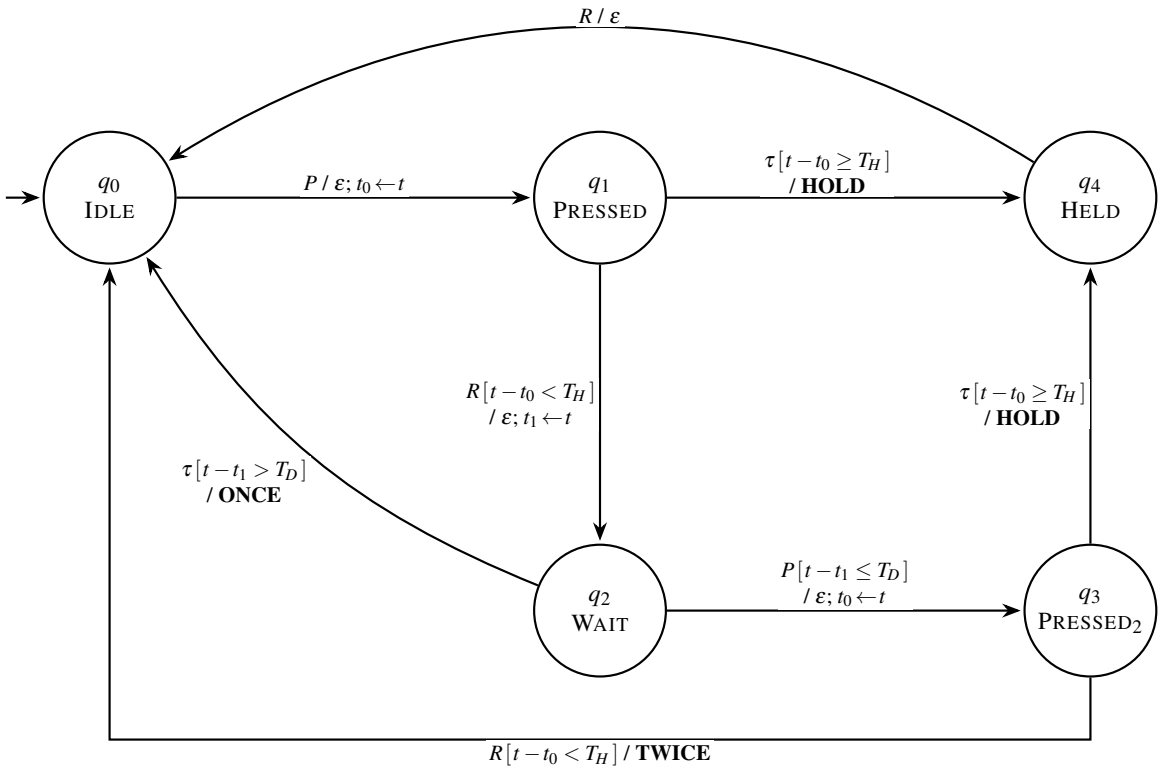
Gambar 1 — Komposisi



Gambar 1: Komposisi antarmuka OLED dan tombol BOOT.

Komposisi antarmuka: pengenal kejadian tombol M_B mengubah level GPIO9 menjadi kejadian ONCE/TWICE/HOLD yang menjadi masukan statechart M_{UI}; M_{UI} juga menerima kejadian dari klien MQTT dan menggerakkan layar OLED serta aplikasi (telemetri, NVS).

Gambar 2 — Pengenal kejadian tombol M_B



Gambar 2: Pengenal kejadian tombol M_B (FSM Mealy berwaktu).

Definisi formal. $M_B = (Q, \Sigma, \Lambda, C, \delta, \lambda, q_0)$ dengan

- $Q = \{q_0\,\text{Idle},\ q_1\,\text{Pressed},\ q_2\,\text{Wait},\ q_3\,\text{Pressed}_2,\ q_4\,\text{Held}\}$
- $\Sigma = \{P, R, \tau\}$: tepi tekan, tepi lepas (keduanya ter-*debounce*: sebuah perubahan level diterima hanya bila $t - t_c \geq T_{DB}$, t_c = waktu perubahan terakhir yang diterima), dan detak waktu
- $\Lambda = \{\text{ONCE}, \text{TWICE}, \text{HOLD}, \varepsilon\}$
- $C = \{t_0, t_1\}$: waktu tekan terakhir dan lepas terakhir
- $T_{DB} = 30$ ms, $T_H = 700$ ms, $T_D = 350$ ms

Tabel transisi (pasangan yang tidak tercantum: tetap di keadaan yang sama, keluaran ε)

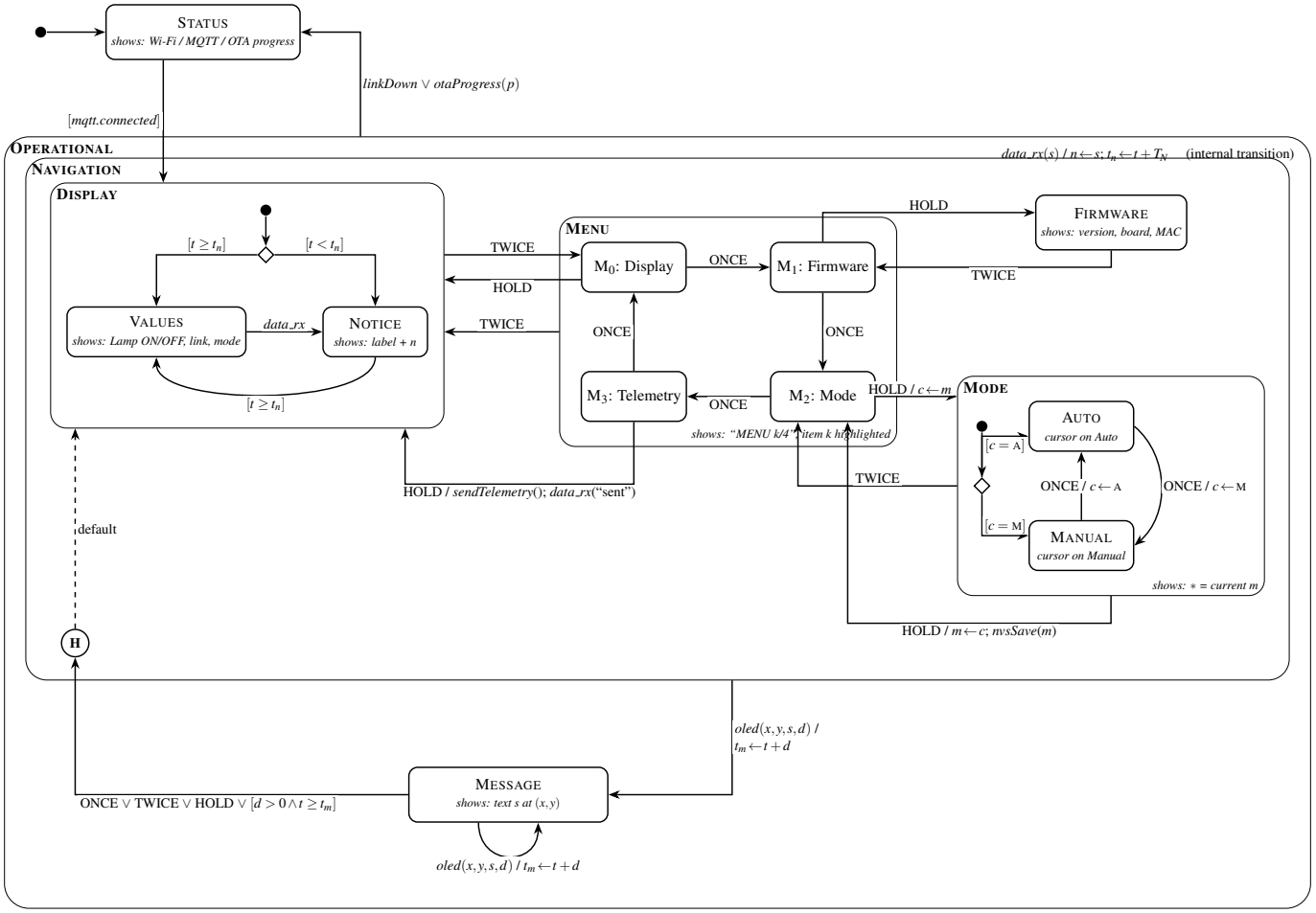
Keadaan	Masukan	Penjaga	Keadaan berikut	Keluaran	Aksi
q_0 Idle	P	—	q_1	ε	$t_0 \leftarrow t$
q_1 Pressed	R	$t - t_0 < T_H$	q_2	ε	$t_1 \leftarrow t$
q_1 Pressed	τ	$t - t_0 \geq T_H$	q_4	HOLD	—
q_2 Wait	P	$t - t_1 \leq T_D$	q_3	ε	$t_0 \leftarrow t$
q_2 Wait	τ	$t - t_1 > T_D$	q_0	ONCE	—
q_3 Pressed$_2$	R	$t - t_0 < T_H$	q_0	TWICE	—

Keadaan	Masukan	Penjaga	Keadaan berikut	Keluaran	Aksi
q_3 Pressed$_2$	τ	$t - t_0 \geq T_H$	q_4	HOLD	—
q_4 Held	R	—	q_0	ε	—

Pemetaan ke kode. Keadaan tidak disimpan dalam satu variabel, tetapi dikodekan oleh tiga variabel `readButton()`: $q_0 \equiv$ (`buttonDown`=0, `clicks`=0); $q_1 \equiv$ (1, 0, `holdReported`=0); $q_2 \equiv$ (0, 1); $q_3 \equiv$ (1, 1, `holdReported`=0); $q_4 \equiv$ (1, $\cdot$, `holdReported`=1). t_0 = `pressedAt`, t_1 = `releasedAt`, t_c = `buttonChanged`.

Sifat. Mesin deterministik dan lengkap: penjaga pada setiap keadaan saling lepas ($t - t_0 < T_H$ vs. $\geq T_H$; $t - t_1 \leq T_D$ vs. $> T_D$). Setiap rangkaian tekan menghasilkan tepat satu keluaran bukan-ε; ONCE tertunda T_D setelah dilepas (harga untuk membedakan tekan sekali dari tekan dua kali), TWICE dan HOLD dikeluarkan seketika. Simulasi jejak masukan menghasilkan ONCE pada 455 ms (tekan 0–100 ms), TWICE pada 330 ms, HOLD pada 700 ms, dan *bounce* 10 ms diabaikan.

Gambar 3 — Statechart antarmuka OLED M_{UI}



Events ONCE, TWICE, HOLD: outputs of M_B (Fig. 2)
data_rx(s): desired / command / rule / error received
oled(x,y,s,d): command `oled` {x, y, msg, ms}
linkDown: Wi-Fi/MQTT (re)connecting; *otaProgress*(p)
completion guards $[\cdot]$ are evaluated on every `loop()` pass

Variables c: highlighted mode, $m \in \{\text{A},\text{M}\}$: telemetry mode (NVS)
n: notice text, t_n: notice deadline, t_m: message deadline
Constants $T_N = 2$ s; d defaults to 15 s ($d = 0$: until a press)
Notation *event* [*guard*] / *action*; H: shallow history
Unlisted (state, event) pairs: ignored. Each state's *shows* is its Moore output.

Gambar 3: Statechart antarmuka OLED M_{UI} (notasi UML; H = *shallow history*).

Struktur hierarkis.

- **Status** (di luar **Operational**): layar Wi-Fi/MQTT/OTA. Keadaan awal.
- **Operational** = { **Navigation**, **Message** }, dengan transisi internal *data_rx*(s) / $n \leftarrow s$; $t_n \leftarrow t + T_N$ (pemberitahuan dicatat di keadaan mana pun).
- **Navigation** = { **Display** {Values, Notice}, **Menu** {M_0..M_3}, **Firmware**, **Mode** {Auto, Manual}, H }. H adalah *shallow history*: **Message** kembali ke keadaan Navigation terakhir (`beforeMessage`).

Variabel (bagian *extended* dari EFSM). c pilihan yang disorot di Mode; $m \in \{\mathrm{A}, \mathrm{M}\}$ mode telemetri (disimpan di NVS); n teks pemberitahuan; t_n batas waktu pemberitahuan (T_N = 2 s); t_m batas waktu pesan (`ms`, bawaan 15 s; 0 = sampai tombol ditekan). Indeks menu k (`menuIndex`) dinyatakan langsung sebagai sub-keadaan M_k.

Tabel transisi (pasangan yang tidak tercantum: kejadian diabaikan)

Dari	Kejadian [penjaga] / aksi	Ke
• (awal)	—	Status
Status	[*mqtt.connected*]	Display
Operational	*linkDown* $\vee$ *otaProgress*(p)	Status
Display (masuk)	pilihan: $[t < t_n]$ / $[t \geq t_n]$	Notice / Values
Values	*data_rx*	Notice
Notice	$[t \geq t_n]$	Values
Display	TWICE	M_0
Menu	TWICE	Display
M_k	ONCE	$\mathrm{M}_{(k+1) \bmod 4}$
M_0	HOLD	Display
M_1	HOLD	Firmware
M_2	HOLD / $c \leftarrow m$	Mode (pilihan: $[c = \mathrm{A}] \rightarrow$ Auto, $[c = \mathrm{M}] \rightarrow$ Manual)
M_3	HOLD / *sendTelemetry*(); *data_rx*(“sent”)	Display ($\rightarrow$ Notice)
Firmware	TWICE	M_1
Auto	ONCE / $c \leftarrow \mathrm{M}$	Manual
Manual	ONCE / $c \leftarrow \mathrm{A}$	Auto
Mode	HOLD / $m \leftarrow c$; *nvsSave*(m)	M_2
Mode	TWICE	M_2
Navigation	*oled*(x, y, s, d) / $t_m \leftarrow t + d$	Message (H dicatat)
Message	*oled*(x, y, s, d) / $t_m \leftarrow t + d$	Message
Message	ONCE $\vee$ TWICE $\vee$ HOLD $\vee$ $[d > 0 \wedge t \geq t_m]$	H

Keluaran. Isi layar adalah keluaran Moore (fungsi dari keadaan aktif dan variabel, dirender oleh `drawScreen()`); aksi pada transisi (*sendTelemetry*, *nvsSave*) adalah keluaran Mealy. Mode m memengaruhi aplikasi di luar mesin ini: $m = \mathrm{A}$ mengirim telemetri setiap `SEND_INTERVAL_MS`, $m = \mathrm{M}$ hanya lewat M_3.

Catatan pemodelan.

1. Telemetry tidak dimodelkan sebagai keadaan: memilihnya adalah aksi sesaat yang langsung kembali ke Display, sesuai perilaku kode.
2. Pemberitahuan Display dimodelkan dengan penjaga waktu t_n, bukan antrean: kejadian *data_rx* yang datang ketika layar lain aktif tetap dicatat dan tampil bila Display dibuka sebelum t_n.
3. Pada Display dan Firmware, ONCE dan HOLD diabaikan (hanya TWICE yang berlaku), sesuai ketentuan “tekan dua kali = kembali”.
4. Bila `OLED` bernilai `false`, M_B dan M_{UI} tidak dikompilasi; telemetri selalu otomatis.

Contoh keterangan gambar untuk naskah (IEEE).

- *Fig. 1. Composition of the on-device user interface: a timed Mealy machine M_B classifies BOOT-button presses into ONCE, TWICE and HOLD events that drive the UI statechart M_{UI}, together with MQTT events.*
- *Fig. 2. Button event recognizer M_B (timed Mealy machine). Transitions are labelled input [guard] / output; action, with $T_{DB} = 30$ ms, $T_H = 700$ ms and $T_D = 350$ ms.*
- *Fig. 3. Hierarchical statechart of the OLED user interface M_{UI} (UML notation). H denotes shallow history; each state's "shows" annotation is its Moore output.*